Solution: Factor Combinations

Problem Statement

Numbers can be regarded as the product of their factors.

For example, 8 = 2 x 2 x 2 = 2 x 4.

Given an integer n, return all possible combinations of its factors. You may return the answer in **any order**.

Example 1:

```
Input: n = 8
Output: [[2, 2, 2], [2, 4]]
```

Example 2:

```
Input: n = 20
Output: [[2, 2, 5], [2, 10], [4, 5]]
```

Solution

We can use backtracking to find all the factors of a given number n.

We will start by iterating through all integers from start (2 by default) to the square root of $n + 1$. If the current integer `i` divides `n`, we will add `i` to the list of current factors `curr` and appends this list along with the corresponding factor of n/i to the list of all factors (`result`). Then we can recursively call the function with n/i as the new input, `i` as the new start value, and `curr` and `result` as the current factors and results. After each recursive call, we have to pop the last element from `curr` to backtrack and find other factors.

Code

Here is what our algorithm will look like:

Java

```java
import java.util.ArrayList;
import java.util.List;

class Solution {
  public List<List<Integer>> getAllFactors(int n, int start, List<Integer> curr, List<Li
    // This loop is used to find all the factors of the input number n
    for (int i = start; i <= (int)Math.sqrt(n); i++) {
      // If n is divisible by i, add i to the curr list of factors; curr is used to stor
      if (n % i == 0) {
        curr.add(i); // Found a factor, append it to the list of factors
        // Append the current factors and the corresponding factor of n // i to the resu
        List<Integer> currCopy = new ArrayList<>(curr);
        currCopy.add(n/i);
        result.add(currCopy);
        // Recursively call the function with n // i as the new input, i as the new star
        getAllFactors(n / i, i, curr, result);
        curr.remove(curr.size() - 1); // Remove last element from curr to backtrack and
      }
    }
    return result;
  }
```

Run Code Result Stdout

You must run your code first

Execute

Time Complexity

We can't have more than $O(n)$ factors of a number `n`. This means that `getAllFactors` can be called a maximum of $O(n)$ times recursively. The `for` loop iterates a maximum of $O(sqrt(n))$. This means that the overall time complexity is $O(n * sqrt(n))$ or $(n^{1.5})$

Space Complexity

Ignoring the space needed for the output array, the space complexity will be $O(log(n))$ as we need to save only one factor while in the recursion, and our recursion tree won't get bigger than $O(log(n))$ steps.

← Previous
Factor Combinations (Medium)

Next →
Split A String Into The Max Number Of Unique Substrings (Medium)

☐ Mark as Completed